

SIMATS ENGINEERING

CO5- ASSESSMENT TOOL 2– Technical Presentation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 10%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I **Shaik Abdul Razak (192524358)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Abdul Razak

1. System Analysis Presentation

Present a reverse engineering analysis of an AI-based system, explaining its architecture, components, and working process using diagrams.

2. Architecture Reconstruction Demo

Prepare a technical presentation to reconstruct the architecture of a given software/AI system, highlighting module interactions and data flow.

3. Algorithm & Logic Identification

Present how you identified algorithms and decision-making logic in a reverse-engineered system, including methodology and tools used.

4. Data Flow & Processing Pipeline

Explain through a presentation the data flow, processing pipeline, and communication mechanisms in a reverse-engineered distributed system.

5. Enhancement & Redesign Proposal

Deliver a technical presentation proposing improvements or redesign strategies for an existing system based on reverse engineering findings.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

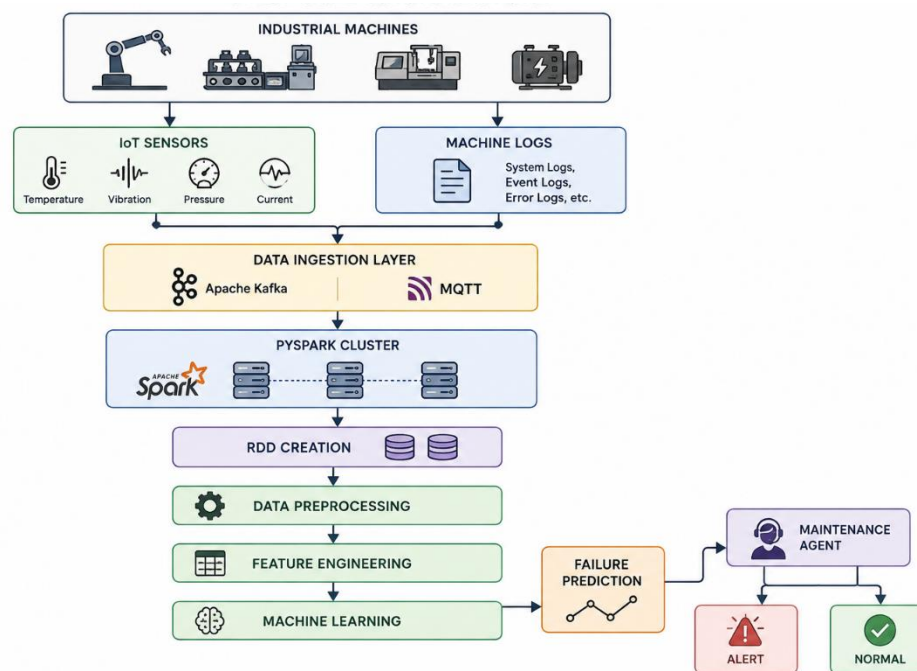
1. SYSTEM ANALYSIS PRESENTATION

AI-Based Predictive Maintenance System

Problem Statement

Industrial machines continuously generate sensor data such as temperature, vibration, pressure, speed, and operating hours. An AI-based predictive maintenance system can analyze this data and identify machines that may fail before an actual breakdown occurs. The objective is to reverse engineer the system and understand its architecture, processing stages, and prediction mechanism.

Proposed Architecture



Main Components

1. Sensor Layer – collects machine measurements.
2. Ingestion Layer – receives continuous sensor information.
3. PySpark Layer – performs distributed processing.
4. RDD Processing Layer – cleans and transforms records.
5. Feature Engineering – calculates useful machine characteristics.
6. ML Model – predicts machine failure.
7. Maintenance Agent – converts predictions into maintenance decisions.

RDD Operations

`filter()` - remove invalid sensor records
`map()` -transform sensor measurements
`reduceByKey()` -calculate machine statistics
`groupByKey()` - group data by machine
`cache()` - reuse frequently accessed data

Working

Raw machine readings are collected and converted into RDDs. PySpark distributes the records among worker nodes. Invalid readings are removed using `filter()`, while `map()` converts raw readings into useful features. Machine-level statistics are generated using `reduceByKey()`. The resulting features are supplied to an ML model that estimates failure probability.

Result

The reconstructed system can identify abnormal machine behaviour and generate early maintenance alerts, reducing unexpected equipment failures and downtime.

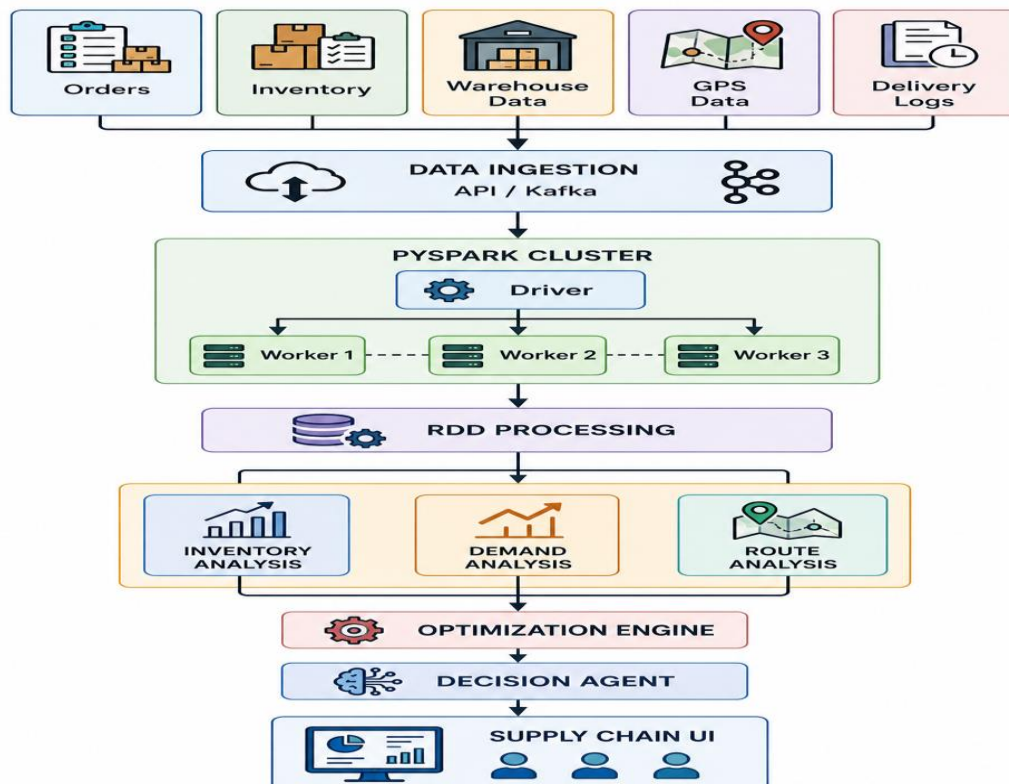
2. ARCHITECTURE RECONSTRUCTION DEMO

AI-Based Supply Chain Optimization

Problem Statement

A supply chain generates information from orders, inventory systems, warehouses, delivery vehicles, and GPS devices. Processing all this information centrally becomes difficult when the dataset becomes large. The objective is to reconstruct a distributed architecture that uses PySpark to process these datasets and support supply-chain decisions.

Proposed Architecture



Module Interaction

Order Module provides customer demand.

Inventory Module provides available stock.

Warehouse Module provides storage information.

GPS Module provides vehicle locations.

PySpark combines these datasets and generates an integrated view for optimization.

Example RDD Processing

```
Order RDD
  +
Inventory RDD
  +
GPS RDD
  ↓
  join()
  ↓
Combined Supply Chain RDD
  ↓
reduceByKey()
  ↓
Warehouse / Route Statistics
```

Working

The system first collects data from different supply-chain sources. PySpark distributes the datasets across worker nodes. RDD transformations clean, aggregate, and combine the information. The optimization module then identifies suitable warehouses, inventory levels, and delivery routes.

Result

The reconstructed architecture provides scalable processing and supports faster supply-chain decisions even when order and logistics data increases significantly.

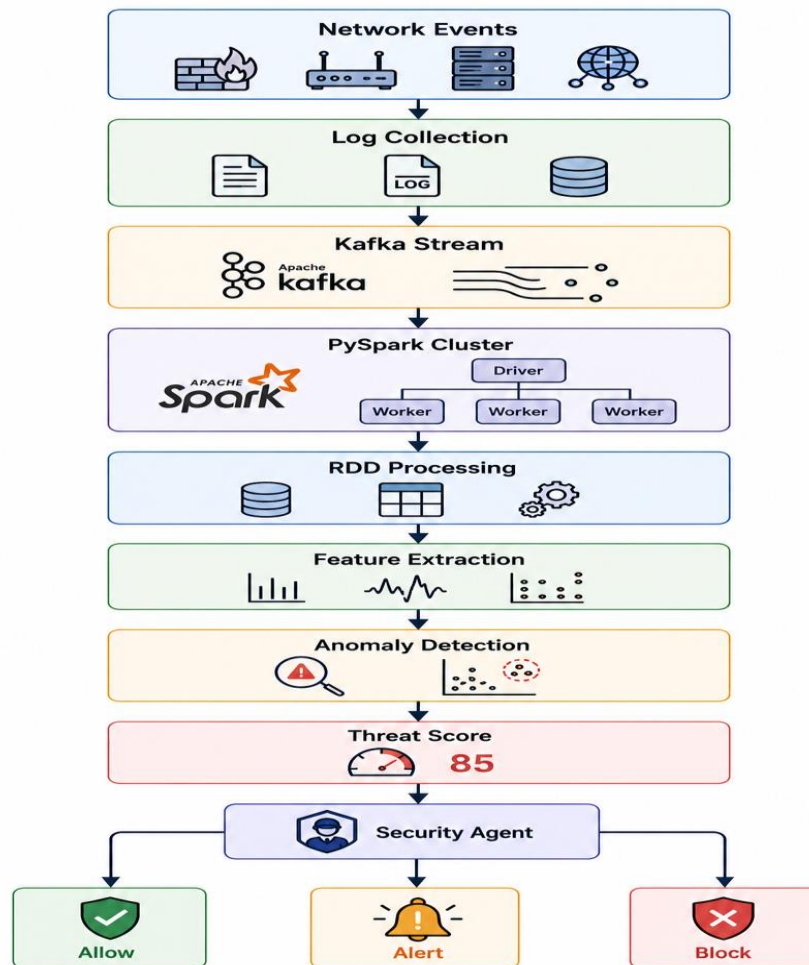
3. ALGORITHM & LOGIC IDENTIFICATION

AI-Based Cybersecurity Threat Detection

Problem Statement

Modern systems generate millions of network events. Detecting suspicious behavior manually is difficult. The objective is to reverse engineer a cybersecurity AI system and identify the processing algorithm and decision logic responsible for detecting abnormal network activity.

Proposed Architecture



Features Analyzed

The system can analyze:

- Number of login attempts
- Number of unique IP addresses
- Request frequency
- Access time
- Failed authentication count
- Data transfer volume

Algorithm Identification Method

System Observation

|

Collect Input/Output

|

Change Input Features

|

Observe Decision Changes

|

Identify Important Features

|

Test Candidate Algorithms

|

Compare Results

|

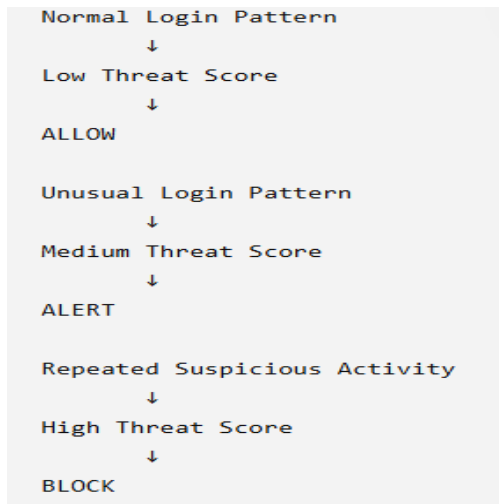
Identify Probable Algorithm

Possible Algorithms

The reverse-engineering process can compare:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Means
- Isolation Forest
- Other anomaly-detection techniques

Example Decision Logic



Result

The analysis identifies the relationship between input network features, anomaly scores, and final security decisions. This allows the probable detection algorithm and decision logic to be reconstructed.

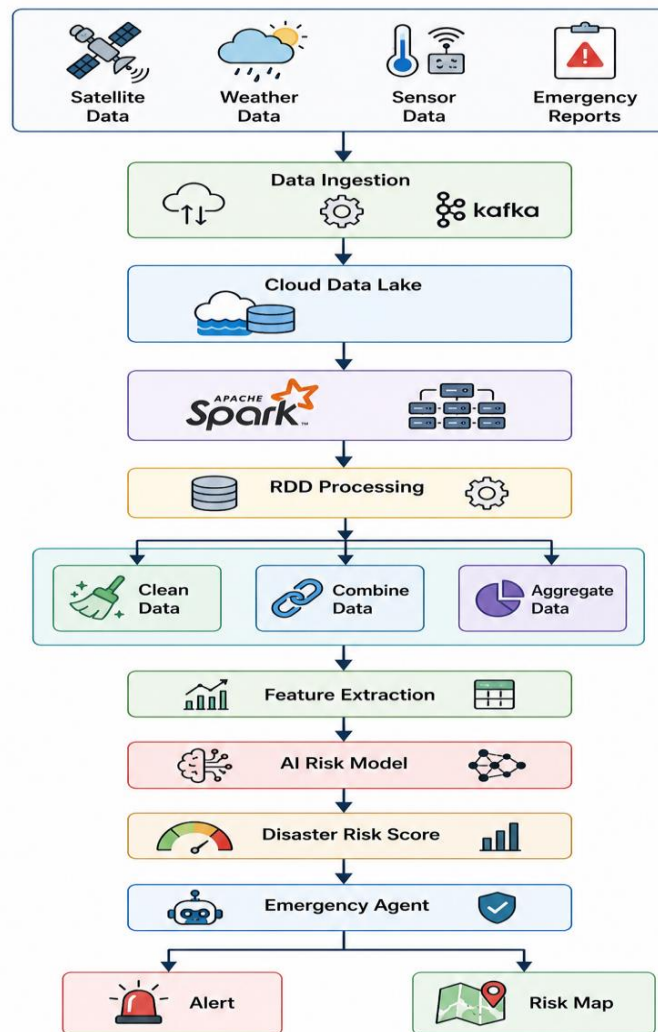
4. DATA FLOW & PROCESSING PIPELINE

AI-Based Disaster Intelligence System

Problem Statement

Disaster monitoring requires information from different sources such as satellites, weather stations, sensors, and emergency reports. The challenge is to combine these heterogeneous datasets and process them quickly to identify potential disaster risks.

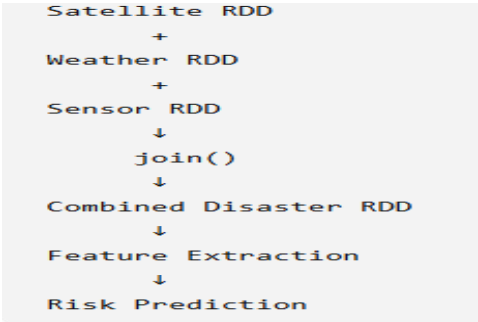
Proposed Architecture



Data Processing

The different sources produce different types of information. PySpark first converts the data into distributed datasets.

For example:



RDD Operations

Operation	Purpose
map()	Convert raw records
filter()	Remove invalid readings
flatMap()	Expand complex reports
join()	Combine different datasets
reduceByKey()	Calculate regional statistics
cache()	Reuse processed data

Working

The system receives data from multiple sources and stores it in a cloud data lake. PySpark processes the data in parallel. Weather, satellite, and sensor information are combined to generate regional risk features. The AI model estimates disaster risk and the emergency agent generates alerts when the risk crosses a specified threshold.

Result

The system provides a distributed pipeline capable of combining multiple data sources and producing timely disaster-risk information for emergency response.

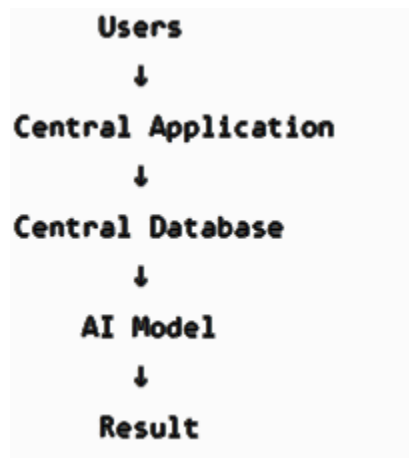
5. ENHANCEMENT & REDESIGN PROPOSAL

Cloud-Based Distributed AI Platform

Problem Statement

The existing AI application uses centralized processing. As the amount of data increases, the central server becomes a bottleneck. The objective is to redesign the system using PySpark, cloud infrastructure, RDD processing, and intelligent agents.

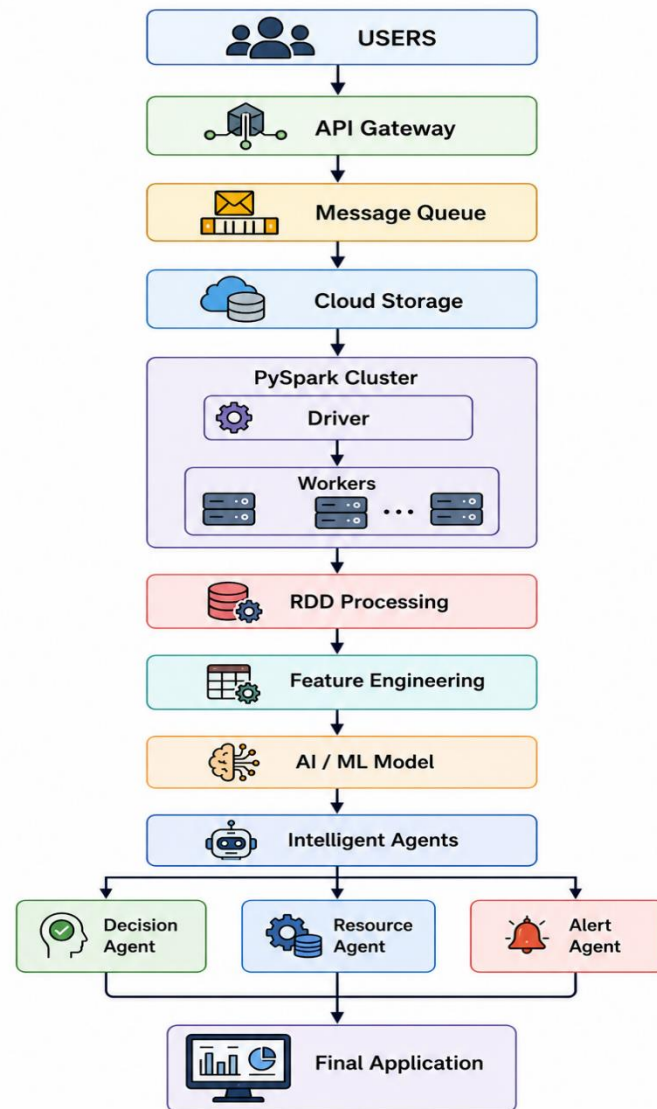
Existing Architecture



Identified Problems

- Central processing bottleneck
- Limited scalability
- High processing time
- Poor fault tolerance
- Manual resource management
- Difficulty handling large datasets

Proposed Architecture



Intelligent Agents

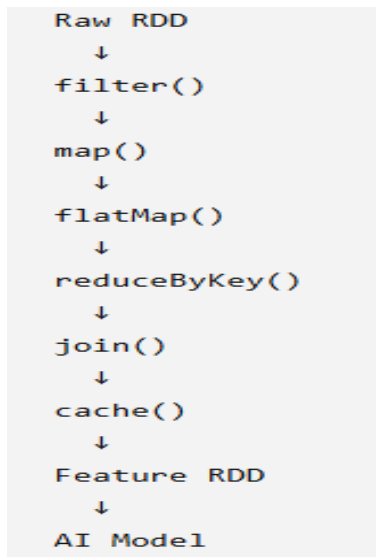
Decision Agent: evaluates AI predictions and selects actions.

Resource Agent: monitors workload and recommends resource scaling.

Alert Agent: generates alerts for abnormal events.

Learning Agent: uses previous outcomes to improve future decisions.

RDD Processing



Existing vs Proposed

Feature	Existing	Redesigned
Processing	Centralized	Distributed PySpark
Data Handling	Limited	Large-scale
Scalability	Low	High
Fault Tolerance	Limited	RDD lineage/recovery
Resource Management	Manual	Agent-assisted
AI Decisions	Fixed	Adaptive
Architecture	Centralized	Modular
Deployment	Single environment	Cloud-based

Result

The redesigned system distributes computation across multiple workers, improves scalability, and reduces the dependency on a single processing server. Intelligent agents provide automated decisions and resource management.